

Robot suiveur de ligne

Commande d'arrêt obstacle via MQTT - HiveMQ WebSocket Client

Objectif	Faire communiquer une simulation Pygame avec un broker MQTT. Le robot suit une ligne noire et s'arrête si le message MQTT indique un obstacle.
Messages MQTT	Obstacle detecte = arrêt du robot Pas d'obstacle = reprise du suivi de ligne
Topic	robot/obstacle

Livrable : simulation fonctionnelle + code Python complet

1. Objectif du TP

Dans ce TP, l'apprenant realise une simulation de robot suiveur de ligne avec Python et Pygame. Le robot suit une ligne noire automatiquement. En parallele, il ecoute un topic MQTT pour recevoir l'etat d'obstacle.

Regle de commande

Si le message MQTT est Obstacle detecte, le robot s'arrete. Si le message est Pas d'obstacle, le robot continue son suivi de ligne.

2. Prerequis

Element	Details
Python	Version recente installee sur le PC.
Bibliotheques	pygame et paho-mqtt.
MQTT	HiveMQ WebSocket Client pour publier les messages.
Broker	broker.hivemq.com - port 1883 cote Python.

Installation des bibliotheques :

```
pip install pygame
pip install paho-mqtt
```

3. Architecture du systeme

```
HiveMQ WebSocket Client
|
| Publish : Obstacle detecte / Pas d'obstacle
v
broker.hivemq.com
|
| Topic : robot/obstacle
v
Programme Python Pygame
|
v
Robot suiveur de ligne
```

Principe

Le robot ne lit pas une distance numerique. Il lit directement un etat texte envoye par MQTT : obstacle detecte ou pas d'obstacle.

4. Deroulement du TP - Steps

Step 1 - Creer le fichier Python

Creer un fichier nomme robot_mqtt.py. Ce fichier contiendra la simulation, la logique robot et la connexion MQTT.

```
import pygame
import math
import paho.mqtt.client as mqtt
```

Step 2 - Configurer MQTT

Definir le broker HiveMQ, le port et le topic ecoute par le robot.

```
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = "robot/obstacle"
```

Step 3 - Initialiser l'etat obstacle

La variable obstacle_detected decide si le robot peut bouger ou non.

```
obstacle_detected = False
mqtt_connected = False
mqtt_message = "Pas d'obstacle"
```

Step 4 - Lire les messages MQTT

Quand un message arrive, le programme compare son contenu avec les deux commandes attendues.

```
if message == "Obstacle detecte":
    obstacle_detected = True
elif message == "Pas d'obstacle":
    obstacle_detected = False
```

Step 5 - Creer la fenetre Pygame

La fenetre represente l'espace de simulation du robot.

```
WIDTH, HEIGHT = 900, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
```

Step 6 - Garder le meme robot

Le robot est represente par un cercle bleu, deux roues, une direction avant et deux capteurs de ligne.

```
robot_x = 120
robot_y = 300
robot_angle = 0
base_speed = 2.5
turn_speed = 0.06
```

Step 7 - Garder la meme ligne

La ligne noire est le chemin que le robot doit suivre. Les points restent les memes.

```
line_points = [
    (50, 300), (180, 300), (280, 220), (420, 220),
    (540, 360), (680, 360), (800, 250), (880, 250)
]
```

Step 8 - Lire les capteurs

Chaque capteur regarde le pixel sous lui. Si le pixel est sombre, il detecte la ligne noire.

```
left_detect = sensor_detects_line(left_sensor_pos)
right_detect = sensor_detects_line(right_sensor_pos)
```

Step 9 - Bloquer le robot si obstacle

Si MQTT envoie Obstacle detecte, la vitesse devient 0 et le robot reste immobile.

```
if obstacle_detected:
    speed = 0
```

Step 10 - Continuer si pas d'obstacle

Si MQTT envoie Pas d'obstacle, le robot reprend sa logique de suivi de ligne.

```
else:
    if left_detect and right_detect:
        speed = base_speed
    elif left_detect and not right_detect:
        speed = base_speed * 0.8
        robot_angle -= turn_speed
    elif right_detect and not left_detect:
        speed = base_speed * 0.8
        robot_angle += turn_speed
```

Step 11 - Tester sur HiveMQ

Dans la zone Publish de HiveMQ, utiliser le topic et envoyer les messages ci-dessous.

```
Topic : robot/obstacle
Message 1 : Obstacle detecte
Message 2 : Pas d'obstacle
```

Step 12 - Valider le resultat

Le robot doit s'arreter quand obstacle est detecte et reprendre son mouvement quand le message indique qu'il n'y a pas d'obstacle.

5. Procedure de test avec HiveMQ

Dans HiveMQ WebSocket Client, utiliser la partie Publish. Le topic doit etre exactement le meme que dans le code Python.

```
Topic : robot/obstacle  
  
Message pour arreter :  
Obstacle detecte  
  
Message pour continuer :  
Pas d'obstacle
```

Resultats attendus

Message Obstacle detecte : le robot s'arrete et une alerte rouge apparait. Message Pas d'obstacle : le robot continue le suivi de ligne.

6. Questions de comprehension

1. Quel est le role du topic MQTT ?
2. Quelle variable bloque le mouvement du robot ?
3. Pourquoi la vitesse devient 0 quand un obstacle est detecte ?
4. Quelle difference entre Obstacle detecte et Pas d'obstacle ?
5. Pourquoi le robot utilise deux capteurs pour suivre la ligne ?

7. Code complet

Copier ce code dans le fichier robot_mqtt.py, puis lancer le programme avec Python.

```
import pygame
import math
import paho.mqtt.client as mqtt

pygame.init()

# =====
# CONFIGURATION MQTT
# =====
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = "robot/obstacle"

obstacle_detected = False
mqtt_connected = False
mqtt_message = "Pas d'obstacle"

def on_connect(client, userdata, flags, rc):
    global mqtt_connected

    if rc == 0:
        mqtt_connected = True
        print("MQTT connecte")
        client.subscribe(MQTT_TOPIC)
        print("Topic ecoute :", MQTT_TOPIC)
    else:
        mqtt_connected = False
        print("Erreur MQTT :", rc)

def on_disconnect(client, userdata, rc):
    global mqtt_connected
    mqtt_connected = False
    print("MQTT deconnecte")

def on_message(client, userdata, msg):
    global obstacle_detected, mqtt_message

    message = msg.payload.decode().strip()
    mqtt_message = message

    print("Message MQTT reçu :", message)

    if message == "Obstacle detecte":
        obstacle_detected = True

    elif message == "Pas d'obstacle":
        obstacle_detected = False

    else:
        print("Message inconnu. Envoyer : Obstacle detecte ou Pas d'obstacle")

# Client MQTT compatible anciennes et nouvelles versions de paho-mqtt
try:
    mqtt_client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1)
except Exception:
    mqtt_client = mqtt.Client()

mqtt_client.on_connect = on_connect
mqtt_client.on_disconnect = on_disconnect
mqtt_client.on_message = on_message

try:
    mqtt_client.connect(MQTT_BROKER, MQTT_PORT, 60)
    mqtt_client.loop_start()
```

```

except Exception as e:
    print("Connexion MQTT impossible :", e)

# =====
# FENETRE PYGAME
# =====
WIDTH, HEIGHT = 900, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Robot Suiveur de Ligne - MQTT Obstacle")

clock = pygame.time.Clock()

# Couleurs
WHITE = (245, 245, 245)
BLACK = (20, 20, 20)
DARK = (35, 35, 45)
BLUE = (40, 120, 255)
RED = (230, 60, 60)
GREEN = (50, 180, 90)
GRAY = (180, 180, 180)
LIGHT_GRAY = (235, 235, 235)
PANEL = (255, 255, 255)

# Robot
robot_x = 120
robot_y = 300
robot_angle = 0

robot_radius = 22
base_speed = 2.5
turn_speed = 0.06

# Capteurs suiveur de ligne
sensor_distance = 28
sensor_side_offset = 14

# Ligne noire : meme chemin
line_points = [
    (50, 300),
    (180, 300),
    (280, 220),
    (420, 220),
    (540, 360),
    (680, 360),
    (800, 250),
    (880, 250)
]

line_width = 28

# =====
# FONCTIONS GRAPHIQUES
# =====
def draw_line():
    pygame.draw.lines(screen, BLACK, False, line_points, line_width)

def draw_rounded_rect(surface, color, rect, radius=16):
    pygame.draw.rect(surface, color, rect, border_radius=radius)

def draw_text(text, x, y, size=22, color=BLACK, bold=False):
    font = pygame.font.SysFont("Arial", size, bold=bold)
    img = font.render(text, True, color)
    screen.blit(img, (x, y))

```

```

def get_sensor_position(x, y, angle, side):
    """
    side = -1 : capteur gauche
    side = 1  : capteur droit
    """
    front_x = math.cos(angle) * sensor_distance
    front_y = math.sin(angle) * sensor_distance

    side_x = math.cos(angle + math.pi / 2) * sensor_side_offset * side
    side_y = math.sin(angle + math.pi / 2) * sensor_side_offset * side

    sx = x + front_x + side_x
    sy = y + front_y + side_y

    return int(sx), int(sy)

def sensor_detects_line(sensor_pos):
    x, y = sensor_pos

    if x < 0 or x >= WIDTH or y < 0 or y >= HEIGHT:
        return False

    color = screen.get_at((x, y))[:3]

    # Pixel sombre = ligne noire
    return color[0] < 80 and color[1] < 80 and color[2] < 80

def draw_robot(x, y, angle, left_sensor, right_sensor):
    # Halo rouge si obstacle
    if obstacle_detected:
        pygame.draw.circle(screen, RED, (int(x), int(y)), robot_radius + 10, 3)

    # Corps robot
    pygame.draw.circle(screen, BLUE, (int(x), int(y)), robot_radius)

    # Direction avant
    front_x = x + math.cos(angle) * robot_radius
    front_y = y + math.sin(angle) * robot_radius
    pygame.draw.line(screen, WHITE, (x, y), (front_x, front_y), 4)

    # Roues
    left_wheel = (
        int(x + math.cos(angle + math.pi / 2) * 18),
        int(y + math.sin(angle + math.pi / 2) * 18)
    )

    right_wheel = (
        int(x + math.cos(angle - math.pi / 2) * 18),
        int(y + math.sin(angle - math.pi / 2) * 18)
    )

    pygame.draw.circle(screen, GRAY, left_wheel, 6)
    pygame.draw.circle(screen, GRAY, right_wheel, 6)

    # Capteurs
    left_color = GREEN if left_sensor else RED
    right_color = GREEN if right_sensor else RED

    left_pos = get_sensor_position(x, y, angle, -1)
    right_pos = get_sensor_position(x, y, angle, 1)

    pygame.draw.circle(screen, left_color, left_pos, 6)
    pygame.draw.circle(screen, right_color, right_pos, 6)

```



```

def draw_header():
    draw_rounded_rect(screen, DARK, (20, 15, 860, 70), 18)

    draw_text("Robot Suiveur de Ligne", 45, 28, 28, WHITE, True)
    draw_text("Commande obstacle via MQTT", 45, 58, 18, (210, 210, 210))

    badge_color = GREEN if mqtt_connected else RED
    badge_text = "MQTT connecte" if mqtt_connected else "MQTT non connecte"

    pygame.draw.circle(screen, badge_color, (675, 50), 9)
    draw_text(badge_text, 695, 38, 20, WHITE, True)

def draw_mqtt_help():
    draw_rounded_rect(screen, (250, 250, 250), (20, 95, 500, 55), 12)
    pygame.draw.rect(screen, (220, 220, 220), (20, 95, 500, 55), 1, border_radius=12)

    draw_text("Topic MQTT : robot/obstacle", 38, 104, 18, DARK, True)
    draw_text("Envoyer : Obstacle detecte | Pas d'obstacle", 38, 128, 15, (80, 80, 80))

def draw_status_panel(left_detect, right_detect):
    draw_rounded_rect(screen, PANEL, (20, 505, 860, 75), 18)
    pygame.draw.rect(screen, LIGHT_GRAY, (20, 505, 860, 75), 2, border_radius=18)

    if obstacle_detected:
        state_color = RED
        state_text = "ARRET"
        reason_text = "Obstacle detecte"
    else:
        state_color = GREEN
        state_text = "EN MARCHE"
        reason_text = "Pas d'obstacle"

    pygame.draw.circle(screen, state_color, (55, 543), 13)
    draw_text(state_text, 80, 522, 26, state_color, True)
    draw_text(reason_text, 80, 552, 17, DARK)

    draw_text("Message MQTT", 310, 518, 18, DARK, True)
    draw_text(mqtt_message, 310, 545, 22, BLACK, True)

    draw_text("Commande robot", 550, 518, 18, DARK, True)

    if obstacle_detected:
        draw_text("Robot bloque", 550, 548, 20, RED, True)
    else:
        draw_text("Robot continue", 550, 548, 20, GREEN, True)

    draw_text("Capteurs ligne", 720, 518, 18, DARK, True)

    left_color = GREEN if left_detect else RED
    right_color = GREEN if right_detect else RED

    pygame.draw.circle(screen, left_color, (735, 555), 8)
    draw_text("G", 750, 545, 18, DARK, True)

    pygame.draw.circle(screen, right_color, (800, 555), 8)
    draw_text("D", 815, 545, 18, DARK, True)

def draw_obstacle_alert():
    draw_rounded_rect(screen, RED, (260, 105, 380, 75), 18)

    draw_text("OBSTACLE DETECTE", 315, 118, 28, WHITE, True)
    draw_text("Robot arrete par MQTT", 340, 150, 18, WHITE)

# =====

```

```

# BOUCLE PRINCIPALE
# =====
running = True

while running:
    screen.fill(WHITE)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Dessiner la ligne avant la lecture des capteurs
    draw_line()

    # Positions capteurs
    left_sensor_pos = get_sensor_position(robot_x, robot_y, robot_angle, -1)
    right_sensor_pos = get_sensor_position(robot_x, robot_y, robot_angle, 1)

    # Lecture capteurs
    left_detect = sensor_detects_line(left_sensor_pos)
    right_detect = sensor_detects_line(right_sensor_pos)

    # =====
    # LOGIQUE ROBOT
    # =====
    if obstacle_detected:
        # Si MQTT envoie "Obstacle detecte", le robot reste immobile
        speed = 0

    else:
        # Si MQTT envoie "Pas d'obstacle", le robot suit la ligne
        if left_detect and right_detect:
            speed = base_speed

        elif left_detect and not right_detect:
            speed = base_speed * 0.8
            robot_angle -= turn_speed

        elif right_detect and not left_detect:
            speed = base_speed * 0.8
            robot_angle += turn_speed

        else:
            speed = base_speed * 0.5
            robot_angle += turn_speed * 1.5

    # Mise a jour position
    robot_x += math.cos(robot_angle) * speed
    robot_y += math.sin(robot_angle) * speed

    # Reset si le robot sort de la fenetre
    if robot_x > WIDTH or robot_x < 0 or robot_y > HEIGHT or robot_y < 0:
        robot_x = 120
        robot_y = 300
        robot_angle = 0

    # Affichage robot
    draw_robot(robot_x, robot_y, robot_angle, left_detect, right_detect)

    # Interface
    draw_header()
    draw_mqtt_help()
    draw_status_panel(left_detect, right_detect)

    if obstacle_detected:
        draw_obstacle_alert()

    pygame.display.flip()

clock.tick(60)

mqtt_client.loop_stop()
mqtt_client.disconnect()
pygame.quit()

```